



```
import A5O
import Closure
set_option autoImplicit false
```

```
/-! # A5O — Deployment closure: theapertures.app as a GatedSystem.
```

Observed 2026-09-09 08:24 UTC against https://theapertures.app (public surface):

- Single execution endpoint: POST /api/issue. /api/execute, /api/run, /api/action → 405. No other execution path is exposed.
- ALLOW receipt (RCT-A5O-INSIDER-BRIEF-2026-09-09, A5O-RCT-1.3.1):
 execution_event = { result: EXECUTED, executed_at 08:24:06.373Z }
 issued_at 08:24:06.363Z → issued < executed.
 execution_event is INSIDE the hashed, signed body:
 SHA-256 recomputed = 17a25138...8729 (MATCH); Ed25519 VALID under
 A5O-SIGN-KEY-ED25519-PROD-2026 (published /api/pubkey).
- DENY receipt (appointment revoked, RC-DENY-REVOKED): no execution_event;
 hash MATCH; Ed25519 VALID.
- Revocation snapshot seq 1, revoked = [] (/api/revocations, /.well-known).

Model: the deployment's Time is the receipt stream; an execution IS an execution_event inside a signed receipt whose decision the gate issued. Under that model, interception holds by construction, and gated_sufficiency (Closure.lean) yields accountability over every execution. -/

```
namespace Apertures
```

```
/-- One issued receipt, as observed: the decision the gate took and whether  
an execution_event is present in the signed body. -/
```

```
structure Receipt where
```

```
  appointment_id : String    -- e.g. "APPT-INSIDER-2026"  
  action : String           -- e.g. "brief"  
  allow : Bool              -- decision = ALLOW  
  executed : Bool           -- execution_event.result = EXECUTED present
```

```
/-- Principal := appointment_id string; Agent := member tier; Action := action id;  
Time := the receipt at which the decision was issued. -/
```

```
abbrev Principal := String  
abbrev Agent := String  
abbrev Action := String  
abbrev Time := Receipt
```

```

-- The six predicates, read off the observed receipt surface.
def C : Principal → Agent → Action → Time → Prop := fun p _ x r => r.appointment_id =
p ∧ r.action = x
def I : Principal → Prop := fun _ => True
def M : Agent → Action → Prop := fun _ _ => True
def S : Agent → Action → Prop := fun _ _ => True
def R : Principal → Agent → Action → Time → Prop := fun _ _ _ r => r.allow = true
def Appointed : Principal → Agent → Action → Time → Prop := fun p _ _ r =>
r.appointment_id = p ∧ r.allow = true

/-- The gate: it enforces (p, a, x) at receipt r iff r is an ALLOW receipt
for p and x. This is the observed /api/issue behaviour. -/
def gate : ADASystem Principal Agent Action Time :=
⟨fun p _ x r => r.appointment_id = p ∧ r.action = x ∧ r.allow = true⟩

/-- Execution, as the deployment exposes it: an execution_event inside a
signed receipt. (Model of the observed surface — see attestation below.) -/
def Executes : Agent → Action → Time → Prop := fun _ x r => r.action = x ∧ r.executed
= true

def inAoC : Action → Unit → Prop := fun _ _ => True

/-- OPERATOR ATTESTATION (the one fact a proof cannot observe): the server
executes only inside an issued ALLOW receipt — no execution occurs without
`execution_event` being written into a signed body whose decision is ALLOW.
Stated here as a hypothesis on the deployment, not an axiom of Lean. -/
def NoSideChannel : Prop :=
  ∀ r : Receipt, r.executed = true → r.allow = true

/-- Under the attestation, the gate intercepts every execution. -/
theorem gate_intercepts (h : NoSideChannel) :
  ∀ (A : Agent) (X : Action) (t : Time) (d : Unit),
    inAoC X d → Executes A X t → ∃ P : Principal, gate.enforces P A X t :=
  fun _ _ _ _ ⟨hx, he⟩ => ⟨r.appointment_id, rfl, hx, h r he⟩

/-- theApertures.app as a GatedSystem. -/
def theApertures (h : NoSideChannel) : @GatedSystem Principal Agent Action Time
Unit Executes inAoC :=
  ⟨gate, gate_intercepts h⟩

```

```

/-- The gate achieves ADA: every ALLOW it issues satisfies O. -/
theorem gate_achievesADA : achievesADA C I M S R Appointed inAoC gate :=
  fun _ _ _ _ _ <hp, hx, ha> => <<hp, hx>, trivial, trivial, trivial, ha, hp, ha>

/-- CLOSED: under the operator attestation, theapertures.app is accountable
over every execution — by gated_sufficiency, no further hypothesis. -/
theorem apertures_accountable (h : NoSideChannel) :
  Accountable C I M S R Appointed Executes inAoC :=
  gated_sufficiency C I M S R Appointed Executes inAoC (theApertures h)
gate_achievesADA

-- Witnesses from the 2026-09-09 08:24 UTC observation.
def allowReceipt : Receipt := <"APPT-INSIDER-2026", "brief", true, true>
def denyReceipt : Receipt := <"APPT-INSIDER-2026", "brief", false, false>

theorem allow_enforced : gate.enforces "APPT-INSIDER-2026" "insider" "brief"
allowReceipt :=
  <rfl, rfl, rfl>
theorem allow_satisfies_O : O C I M S R Appointed "APPT-INSIDER-2026" "insider"
"brief" allowReceipt :=
  gate_achievesADA _ _ _ _ () trivial allow_enforced
theorem deny_not_executed : ¬ Executes "insider" "brief" denyReceipt :=
  fun <_, he> => Bool.noConfusion he

end Apertures

#print axioms Apertures.apertures_accountable
#print axioms Apertures.allow_satisfies_O
#print axioms Apertures.deny_not_executed

```